

CTO_BIBLE

CTO Bible — Pièces

Public : une nouveaue CTO qui reprend Pièces demain matin. **Objectif** : 0 → opérationnel en une journée. Tout ce qu'il faut pour faire tourner la prod, livrer du code, et ne pas casser ce qui marche.

Avant de toucher quoi que ce soit : lis ce fichier en entier, puis `CLAUDE.md` et `DESIGN.md`. Les deux sont load-bearing.

1. Le produit en 90 secondes

Pièces est une **marketplace de pièces auto d'occasion à Abidjan** (Côte d'Ivoire). Tout est en **français**, devise **FCFA**, téléphones au format `+225XXXXXXXX`.

Flux tripartite (cœur du produit) :

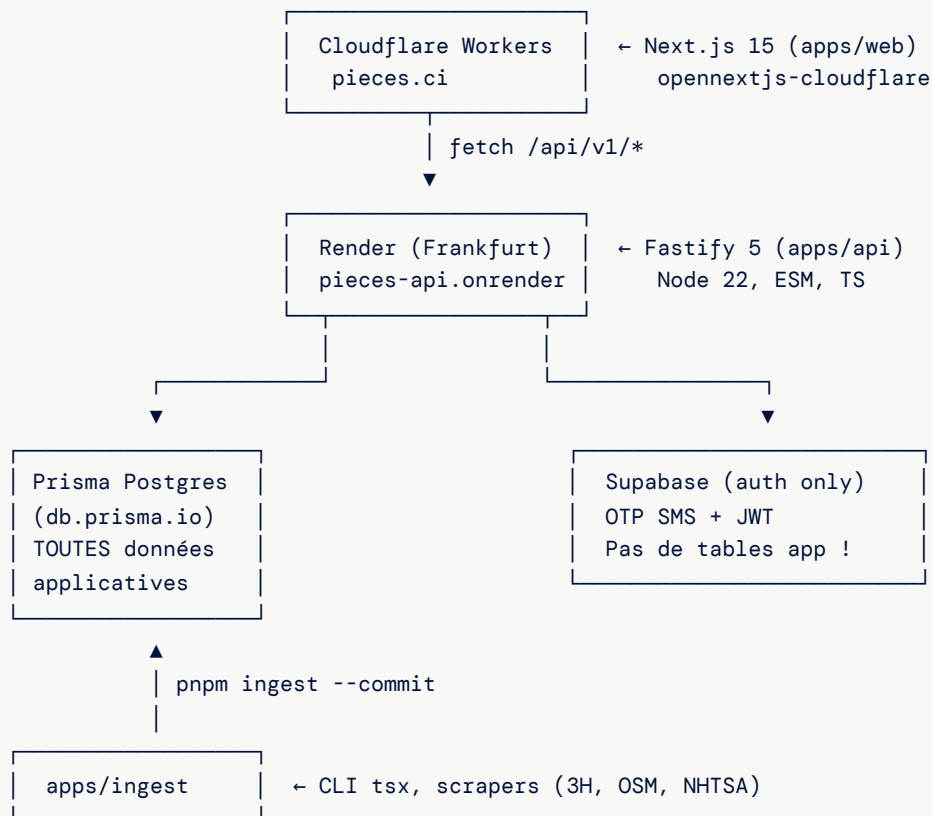
1. Un **mécanicien** identifie une pièce (photo + IA Gemini, ou recherche manuelle).
2. Il crée une commande avec un `shareToken`.
3. Le **propriétaire** du véhicule paie via le lien partagé (Mobile Money escrow CinetPay, ou COD).
4. Un **livreur** récupère chez le **vendeur** et livre.
5. La confirmation libère l'escrow (ou auto-release à 48h après livraison).

6 + 1 rôles : `MECHANIC` (défaut), `OWNER`, `SELLER`, `RIDER`, `ADMIN`, `ENTERPRISE`, plus `LIAISON` (agent terrain qui onboard les vendeurs informels). Un user peut cumuler plusieurs rôles ; `activeContext` détermine le rôle actif côté UI/API.

Trois axes business actuels :

- **B2C** marketplace classique (le flux ci-dessus).
- **B2B Flotte** (`Enterprise`) : abonnements `FREE` / `PRO_FLOTTE` / `PRO_FLOTTE_PLUS` pour gestionnaires de flotte VTC, BTP, logistique.
- **Liaison** : agents terrain qui digitalisent les vendeurs Adjamé / Yopougon (informels, sans smartphone) et publient leur stock dans le catalogue.

2. Architecture en un schéma



Services externes côté API :

- └ CinetPay (paiements Orange Money / MTN / Wave / COD)
- └ WhatsApp Cloud (notifications + bot, HMAC webhook)
- └ Google Gemini (vision IA pour identifier pièces, 2.0-flash)
- └ Cloudflare R2 (stockage images, variants Sharp)
- └ FNE-CI (à venir : factures normalisées)

Split DB critique : Supabase ne contient **QUE l'auth** (utilisateurs / sessions JWT). Toutes les données métier (User, Vendor, Order, etc.) sont dans **Prisma Postgres**. Ne PAS confondre — les liaisons se font via `User.supabaseId`. Voir [memory/db-architecture.md](../memory pour user) si tu as accès.

3. Monorepo — où va quoi

```
pieces/
├── apps/
│   ├── api/                # Fastify backend
│   │   ├── src/
│   │   │   ├── server.ts    ← entrée + plugins
│   │   │   ├── plugins/     ← auth, error handler, multipart, cors, helmet, swagger
│   │   │   ├── lib/         ← prisma, supabase, r2, zodSchema, AppError
│   │   │   └── modules/     ← 1 dossier par domaine métier
│   │   ├── vitest.config.ts
│   │   └── esbuild.config.js
│   ├── web/                # Next.js 15.3 (App Router, React 19, Tailwind 4, PWA Serwist)
│   │   ├── app/
│   │   │   ├── (public)/    ← landing
│   │   │   ├── (auth)/      ← layout protégé + ConsentModal + AppShell
│   │   │   │   ├── admin/   ← 8 sous-pages
│   │   │   │   ├── dashboard/
│   │   │   │   ├── liaison/
│   │   │   │   ├── orders/
│   │   │   │   ├── rider/
│   │   │   │   ├── vendors/
│   │   │   │   ├── vehicles/
│   │   │   │   └── profile/
│   │   │   ├── enterprise/  ← pages publiques B2B Flotte
│   │   │   └── auth/        ← callback Supabase SSR
│   │   ├── components/
│   │   ├── lib/
│   │   ├── next.config.ts   ← rewrites /api/v1/* → API
│   │   └── wrangler.toml    ← Cloudflare Workers (auto-gen via open-next)
│   └── ingest/              # CLI pipeline d'ingestion
│       ├── src/
│       │   ├── cli.ts       ← `pnpm ingest --source 3h --commit`
│       │   ├── sources/     ← three-h.ts, osm.ts, nhtsa.ts
│       │   └── pipeline/    ← load.ts, normalize.ts
│       └── __fixtures__/
├── packages/
│   └── shared/
│       ├── prisma/schema.prisma ← 35+ modèles, source de vérité DB
│       ├── prisma/migrations/
│       ├── env.ts              ← Zod : webEnvSchema + apiEnvSchema (minimal !)
│       ├── validators/        ← Zod partagés (catalog, order, etc.)
│       ├── eslint-config/
│       └── types/
├── docs/                      ← manuels, brochures, deep-dives produits
├── _bmad-output/              ← stories BMAD (sprint-status.yaml)
├── render.yaml                ← déploiement API
├── .github/workflows/         ← ci.yml + deploy-web.yml
├── turbo.json
└── pnpm-workspace.yaml
```

4. Boot local — 15 minutes chrono

Pré-requis

- **Node 22** (`nvm use 22`)
- **pnpm 9+** (`corepack enable`)
- Accès aux secrets: `.env` à la racine + `apps/api/.env` + `apps/web/.env.local`
- Repo à `~/dev/pièces` (PAS dans `~/Documents` — iCloud corrompt les fichiers, voir `memory/repo-location.md`).

Setup

```
cd ~/dev/pièces
pnpm install
pnpm -F shared db:generate      # Prisma Client
pnpm dev                       # lance api + web + ingest en watch
```

- API : `http://localhost:3001` (Swagger sur `/docs`)
- Web : `http://localhost:3000`

Variables d'environnement

Le schéma Zod dans `packages/shared/env.ts` ne valide **qu'un sous-ensemble minimal** (`DATABASE_URL`, `SUPABASE_*`, ports). Les autres clés (CinetPay, Gemini, R2, WhatsApp) sont lues directement via `process.env` dans leurs modules. **Liste complète** :

apps/web/.env.local :

```
NEXT_PUBLIC_API_URL=http://localhost:3001
NEXT_PUBLIC_SUPABASE_URL=https://<project>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_ANON_KEY=eyJ...           # même valeur, ancien nom server-side
NEXT_PUBLIC_SENTRY_DSN=             # optionnel
```

apps/api/.env :

```

# Core
DATABASE_URL=postgres://...          # Prisma Postgres (db.prisma.io)
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
PORT=3001
PINO_LOG_LEVEL=info
SENTRY_DSN=                          # optionnel

# Cloudflare R2 (images catalogue)
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET_NAME=pieces-images
R2_PUBLIC_URL=https://images.pieces.ci

# Google Gemini (vision IA)
GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.0-flash
GEMINI_QUOTA_ALERT_THRESHOLD=0.8

# CinetPay (paiements MM)
CINETPAY_API_KEY=
CINETPAY_SITE_ID=

# WhatsApp Cloud API
WHATSAPP_PHONE_ID=
WHATSAPP_TOKEN=
WHATSAPP_APP_SECRET=
WHATSAPP_VERIFY_TOKEN=pieces-verify-token

```

apps/ingest/.env: `DATABASE_URL=...` (même que l'API).

Vérifier que ça marche

```

curl http://localhost:3001/healthz          # → { ok: true }
curl http://localhost:3001/docs             # Swagger UI
pnpm -F api test                            # vitest
pnpm -F shared db:studio                    # Prisma Studio

```

5. Déploiement

API → Render

- Service: `pieces-api`, région Frankfurt, plan `starter`, Node 22.
- Auto-deploy sur push `main`.
- `render.yaml` au repo root définit tout (build, start, env vars).
- Build: `pnpm install && pnpm -F shared db:generate && pnpm -F api build`
- Start: `pnpm -F shared exec prisma migrate deploy && pnpm -F api start` ← **les migrations passent au démarrage**, attention aux migrations dangereuses.
- Health check: `/healthz` (Render redéploie si non-200).

- Secrets (`sync: false`) à set dans le dashboard Render : `DATABASE_URL` , `SUPABASE_URL` , `SUPABASE_SERVICE_ROLE_KEY` , plus tout R2/Gemini/CinetPay/WhatsApp ajoutés à la main.
- URL prod : `https://pieces-api.onrender.com`

Web → Cloudflare Workers

- Adapter `@opennextjs/cloudflare` (Next 15 → Workers).
- Workflow : `.github/workflows/deploy-web.yml` (push `main` ou manual).
- Secrets GitHub Actions :
 - `secrets.CLOUDFLARE_API_TOKEN`
 - `vars.NEXT_PUBLIC_API_URL` , `vars.NEXT_PUBLIC_SUPABASE_URL` , `vars.NEXT_PUBLIC_SUPABASE_ANON_KEY`
- **Tu ne peux PAS déployer en local depuis macOS 12.6** (workerd exige ≥ 13.5). Pousse sur `main` ou lance le workflow `workflow_dispatch` . Voir `memory/feedback-cloudflare-local-deploy.md` .

CI (`.github/workflows/ci.yml`)

- Lint → Test → Build sur PR et push `main` .
- Tout en parallèle via `turbo run` .

Base de données prod

- Hôte : **Prisma Postgres** (db.prisma.io), pas Supabase (rappel important).
- Migrations : `pnpm -F shared exec prisma migrate deploy` (auto au boot Render).
- Pour exécuter du SQL one-shot : `pnpm -F shared exec prisma db execute --file <fichier.sql> --schema packages/shared/prisma/schema.prisma` .
- **PAS de Supabase MCP pour la prod** — Supabase n'a que l'auth.

6. API — conventions et modules

Pattern de module (à respecter strictement)

Chaque dossier `apps/api/src/modules/<x>/` suit :

- `<x>.routes.ts` — déclaration Fastify + schémas Zod via `zodToFastify()` (de `src/lib/zodSchema.ts`).
- `<x>.service.ts` — logique métier + Prisma.
- `<x>.service.test.ts` / `<x>.routes.test.ts` — vitest.

Règle d'or : les routes n'accèdent **jamais** à Prisma directement. Tout passe par le service.

Plugins globaux

Tous dans `apps/api/src/plugins/` , enregistrés via `fp()` (`fastify-plugin`) :

- `helmet` , `cors` , `rateLimit` , `swagger` , `multipart`
- `auth.ts` — `requireAuth` , `requireRole(...)` , `requireConsent` (preHandlers)
- Error handler global → convertit `AppError` et Zod errors en JSON `{ error: { code, message, statusCode, details } }`

Gotcha critique : si tu enregistres un content-type parser DANS un plugin de route (sans `fp()`), il sera scopé à ce plugin. C'est **intentionnel** pour le webhook WhatsApp (capture raw body pour HMAC SHA-256). Ne pas "corriger" sans

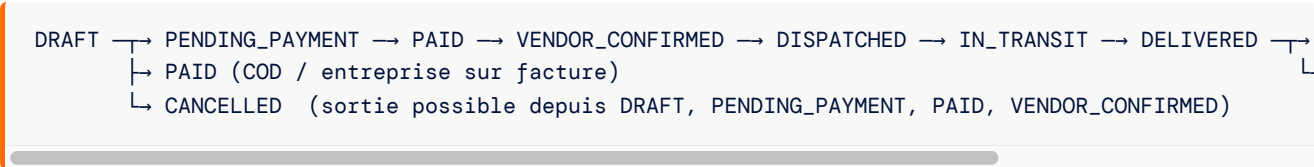
réfléchir.

Liste des modules (`apps/api/src/modules/`)

Module	Rôle
<code>auth</code>	OTP send/verify (Supabase backend)
<code>user</code>	Profil, rôles, activeContext, véhicules, consent
<code>vendor</code>	Onboarding vendeur, KYC (RCCM/CNI), signatures de garantie, zones
<code>catalog</code>	CatalogItem (CRUD), photos, fitments, statuts
<code>browse</code>	Recherche véhicule → modèles → générations → moteurs → pièces
<code>vision</code>	Identification IA Gemini, enqueue Job
<code>order</code>	State machine + CRUD + <code>/share/:token</code> public
<code>payment</code>	CinetPay init + webhook
<code>delivery</code>	Assignment rider, tracking, confirmation
<code>returns</code>	Workflow retour 48h (DEFECTIVE / WRONG_PART / NOT_AS_DESCRIBED)
<code>review</code>	Notes vendeur + livreur
<code>notification</code>	Envoi WhatsApp / SMS sortants
<code>whatsapp</code>	Webhook entrant Cloud API (HMAC)
<code>consent</code>	Tracking RGPD/CCPA-style
<code>liaison</code>	Agent terrain (onboard vendeurs, upload catalogue CSV)
<code>enterprise</code>	Flotte : entreprises, membres, véhicules, abonnements, buffer stock
<code>queue</code>	Worker background jobs (image variants Sharp, IA identification)
<code>admin</code>	Dashboards admin (vendors, clients, finances FNE-CI, etc.)

State machine commande

Fichier: `apps/api/src/modules/order/order.stateMachine.ts`



`canTransition(from, to)` est **OBLIGATOIRE** avant tout `prisma.order.update({ status })` . Toute transition crée un `OrderEvent` (audit).

AppError

```
import { AppError } from '@lib/AppError'
throw new AppError('ORDER_NOT_FOUND', 404, { message: 'Commande introuvable' })
```

Le handler global la transforme en `{ error: { code, message, statusCode, details } }`. Les messages sont en français (user-facing).

Validators partagés

Tout schéma Zod vit dans `packages/shared/validators/`. Tu l'importes côté API ET côté Web (mêmes types via `z.infer<>`).

7. Web — conventions

- **App Router** Next 15, React 19, Tailwind 4 (config dans `apps/web/app/globals.css`).
- **Route groups**: `(public)` = libre, `(auth)` = protégé (middleware redirige vers `/login` si pas de session Supabase).
- **AppShell**: layout commun aux pages auth (header, nav, ConsentModal).
- **API calls**: `fetch('/api/v1/...')` → rewrite dans `next.config.ts` vers `NEXT_PUBLIC_API_URL`.
- **PWA**: Serwist service worker, manifest dans `public/`.

Règles DESIGN.md (load-bearing — voir `DESIGN.md`)

Deux invariants USP à NE PAS casser :

1. **Chips de condition first-class**: chaque produit / order line / admin row affiche la condition (`Neuf` / `Occasion` `importée` / `Ré-usiné` / `Aftermarket` / `OEM`) en chip coloré, jamais en gris noyé dans le texte.
 2. **Breakdown prix explicite**: sur `/choose/[shareToken]`, fiche produit, récap commande → afficher **prix vendeur + main d'œuvre + livraison + frais plateforme + total** avant le bouton "Payer". Zéro frais caché.
-

8. Base de données (résumé)

Schéma complet: `packages/shared/prisma/schema.prisma` (~1000 lignes, 35+ modèles).

Modèles centraux: `User`, `Vendor`, `VendorKyc`, `VendorGuaranteeSignature`, `CatalogItem`, `CatalogItemPhoto`, `CatalogItemFitment`, `Order`, `OrderItem`, `OrderEvent`, `EscrowTransaction`, `Delivery`, `ReturnOrder`, `Dispute`, `SellerReview`, `DeliveryReview`, `Invoice`.

Flotte (B2B): `Enterprise`, `EnterpriseMember`, `EnterpriseSubscription`, `EnterpriseBufferStock`, `EnterpriseMonthlyInvoice`, `Vehicle`, `MaintenanceSchedule`.

Référentiel (ingest): `VehicleMake`, `VehicleModel`, `VehicleGeneration`, `VehicleEngine`, `PartCategory`, `PartReference`, `PartReferenceFitment`, `MarketPriceObservation`, `CompetitorVendor`.

Plateforme: `Job` (background), `ActivityLog` (audit), `DataDeletionRequest` (RGPD), `ConsentRecord`.

Enums clés: `Role`, `OrderStatus`, `VendorStatus`, `CatalogItemStatus`, `PartCondition` (NEW/USED/REFURBISHED), `PartSource` (OEM/AFTERMARKET/COMPATIBLE), `DeliveryStatus`, `PaymentMethod` (ORANGE_MONEY/MTN_MOMO/WAVE/COD), `IngestSource`, `SubscriptionTier` (FREE/PRO_FLOTTE/PRO_FLOTTE_PLUS), `SubscriptionStatus`, `BillingCycle`.

Champ external source: `Vendor.externalSource` + `Vendor.isExternal` permettent les "vendeurs fantômes" créés par l'ingest (1 par source externe via index unique). `CatalogItem.externalSource` + `CatalogItem.externalUrl` pour les pièces scrapées.

9. Intégrations externes — où ça vit

Service	Module	Env vars	Notes
Supabase Auth	auth , lib/supabase.ts , plugins/auth.ts	SUPABASE_URL , SUPABASE_SERVICE_ROLE_KEY	Upsert User à chaque requête. Merge sur email si conflit P2002.
CinetPay	payment	CINETPAY_API_KEY , CINETPAY_SITE_ID	Webhook : POST /api/v1/webhooks/cinetpay . Crée EscrowTransaction(status=HELD) sur paiement confirmé.
WhatsApp Cloud API v18	whatsapp , notification	WHATSAPP_PHONE_ID , WHATSAPP_TOKEN , WHATSAPP_APP_SECRET , WHATSAPP_VERIFY_TOKEN	HMAC SHA-256 vérifié sur raw body. Templates documentés dans docs/whatsapp-templates.md .
Google Gemini 2.0 Flash	vision , queue	GEMINI_API_KEY , GEMINI_MODEL , GEMINI_QUOTA_ALERT_THRESHOLD	Seuils confiance : ≥0.7 identifié, 0.3–0.7 désambiguïsation, <0.3 échec.
Cloudflare R2	catalog , queue , lib/r2.ts	R2_ACCOUNT_ID , R2_ACCESS_KEY_ID , R2_SECRET_ACCESS_KEY , R2_BUCKET_NAME , R2_PUBLIC_URL	S3-compatible. Upload direct + 4 variants Sharp (thumb/small/medium/large).
FNE-CI (factures normalisées)	(à venir) admin/finances	TBD	Invoice a déjà fneValidationNumber , fneQrPayload , fneSubmittedAt . Intégration backoffice pas encore wirée.

10. Ingest — pipeline d'enrichissement

Voir `apps/ingest/` . CLI : `pnpm -F ingest ingest --source <name> [--commit]` .

Sans `--commit` = dry-run (écrit un dump JSON sur disque, n'écrit pas en DB). **Avec `--commit`** = écrit en DB via `loadXxxItems()` .

Sources actuelles :

- `three-h` — 3H Autoparts (parser JSON-LD schema.org, scraper Cheerio).
- `osm` — OpenStreetMap (cartographie vendeurs).
- `nhtsa` — référentiel véhicules US.

Phases (roadmap) :

1.  Référentiel véhicules (NHTSA, Wikipedia, OSM)
2.  Référentiel pièces (catégories, OEM)
3.  Scrapers concurrents (3H done, Jumia / Coin Afrique / Hauto Parts à venir)
4.  Cartographie concurrents (zones, offline)
5.  Observations prix marché continues

Pattern à suivre : voir `apps/ingest/src/sources/competitor.ts` pour le pipeline canonique (fetch → normalize → load DB → vitest avec Prisma mocké).

Gotcha : la CLI 3H avait un bug `dryRun || true` qui forçait toujours dry-run. Fixé fin mai 2026. Toujours tester `--commit` sur staging avant prod.

11. Tests & qualité

- **Framework** : Vitest 3.2 partout.
- **API integration tests** : `buildApp()` depuis `server.ts` + `app.inject()`. Mock pattern : `vi.mock()` AVANT `import`. Env stubbé via `vi.stubEnv()`.
- **Gotcha format FR** : `toLocaleString('fr-FR')` insère un espace insécable U+00A0 entre milliers. Utilise une regex (`toMatch(/4.500 FCFA/)`), pas `toContain` avec espace normal.
- **Lint** : `pnpm lint` (eslint partagé via `packages/shared/eslint-config`).
- **Format** : `pnpm format` (Prettier).
- **Type-check** : `pnpm -F api typecheck`, idem pour web/ingest/shared.

Tout passe en CI sur chaque PR. Un fail bloque le merge.

12. BMAD — workflow de planification

Le repo utilise BMAD (Business Model Agent Development) pour la gestion des stories.

- Story files : `_bmad-output/implementation-artifacts/`
- Sprint tracking : `_bmad-output/sprint-status.yaml`
- Skills invoquées via `/bmad-bmm-*` (`create-story`, `dev-story`, `code-review`, `retrospective`).

Si tu n'aimes pas BMAD, tu peux ignorer — rien ne casse. Mais les stories ouvertes sont la trace de "ce qui était prévu mais pas fini".

13. Pièges à éviter (apprentissages durs)

1. **Ne PAS déplacer le repo dans iCloud** (`~/Documents`, `~/Desktop`). Les fichiers apparaissent "deleted" en git. Le repo est à `~/dev/pièces`.
2. **Ne PAS confondre Supabase et la DB app**. Supabase = auth only. Le MCP Supabase ne sert à rien pour les données métier.
3. **Ne PAS auto-suggérer une commission Liaison basée sur le prix** côté UI. Floor strictement serveur. Voir `memory/feedback-liaison-commission.md`.
4. **Ne PAS skip `fp()`** sur un plugin global Fastify. Sauf cas WhatsApp HMAC raw body (volontaire).
5. **Ne PAS update `Order.status` sans `canTransition()`**. Toute mutation passe par la state machine + crée un `OrderEvent`.
6. **Ne PAS oublier `pnpm db:generate` après modif `schema.prisma`**. CI le fait, mais en local c'est manuel.
7. **Ne PAS amender les migrations Prisma déjà déployées**. Crée une nouvelle migration.
8. **Variables `NEXT_PUBLIC_*`** : injectées au `build` Next.js, pas au runtime. Si tu changes une var sur Cloudflare, il faut redéployer.
9. **Tarifs `--no-verify` et `--force` interdits** sauf cas explicitement justifié.
10. **Cron escrow auto-release 48h** : si le worker plante, escrow restera HELD. Surveiller `Job.status='FAILED'` dans le dashboard admin.

14. Activité récente & priorités probables

Derniers gros chantiers (mai 2026) :

-  Ingest 3H Autoparts (DB load + flag `--commit` explicite)
 -  Champs external source sur `Vendor` et `CatalogItem`
 -  Page Liaison à côté du lien Admin pour les utilisateurs ADMIN
 -  Fallback prod-safe pour le rewrite API web (`pieces-api.onrender.com`)
 -  Module Enterprise (flotte) — abonnements, ROI calculator, buffer stock UI
 -  Pricing flotte 2026-05 (docs commerciaux)
 -  Scrapers Jumia, Coin Afrique, Hauto Parts (phase 3 suite)
 -  Submission FNE-CI (factures normalisées CI) — schema prêt, intégration à wirer
 -  Auto-replenish buffer stock entreprise (logique à implémenter, UI déjà là)
-

15. Aide-mémoire commandes

```
# Dev
pnpm dev                                # tout en watch
pnpm -F api dev                          # API seule
pnpm -F web dev                          # web seule (turbopack)
pnpm -F ingest ingest --source 3h       # ingest dry-run
pnpm -F ingest ingest --source 3h --commit # ingest écrit en DB

# Build / test / lint
pnpm build
pnpm test
pnpm lint
pnpm format

# DB
pnpm -F shared db:generate
pnpm -F shared db:migrate           # crée + applique migration dev
pnpm -F shared db:push              # quick sync (proto)
pnpm -F shared db:studio            # GUI

# Test ciblé
cd apps/api && pnpm vitest run src/modules/order/order.service.test.ts

# Prod SQL one-shot
pnpm -F shared exec prisma db execute --file scripts/foo.sql --schema
  packages/shared/prisma/schema.prisma

# Logs Render API
# → dashboard Render → pieces-api → Logs (Pino JSON, niveau via PINO_LOG_LEVEL)
```

16. Documentation produit complémentaire

Dans `docs/` :

- `project-overview.md` , `architecture-api.md` , `architecture-web.md` , `data-models.md` , `integration-architecture.md` , `api-contracts.md`
- `development-guide.md` , `source-tree-analysis.md`
- Manuels deep-dive par persona : mécanicien, vendeur, propriétaire, livreur, admin, entreprise, liaison, paiement escrow, vision IA, bot WhatsApp, visiteur.
- Brochures commerciales B2B (VTC, BTP, flotte) — docs/marketing.

Et à la racine :

- `CLAUDE.md` — instructions pour les agents IA, mais reste un excellent résumé exécutif.
- `DESIGN.md` — système de design (load-bearing).
- `Guide-Test-Developpement-Pieces.md` — guide QA fonctionnel par scénarios.

17. Contacts & comptes

- **Owner produit** : Fernando Kouame — fernando.kouame@gmail.com (préfère le français, réponses pragmatiques et terSES).
- **Repo GitHub** : voir `git remote -v`.
- **Hébergeurs** : Render (API), Cloudflare (Web), Prisma Cloud (DB), Supabase (auth).
- **Domaine** : `pieces.ci` (à confirmer en console DNS).

Bon courage. Lis le code, respecte la state machine, garde les chips de condition. Le reste se devine.